

Problem-solving agents

Solving problems by searching

Course content

Problem solving in simple environments

- Formulating problems
- Search algorithms
- Search strategies
 - Informed Strategies
 - ▶ the agent can estimate how far it is from the goal
 - Uninformed strategies
 - ▶ no estimation available

Intelligent agents (recap)

- AI (modern view): The study and construction of **rational agents**
rational = **act** rationally = achieve the **best outcome**
- Rationality can be defined mathematically: select an action that maximizes its **utility** (a.k.a. **performance measure**), given
 - 1 the evidence provided by the percept sequence received so far and
 - 2 whatever built-in knowledge the agent has.

Some important kinds of agents:

- 1 Simple reflex agents
- 2 Model-based reflex agents
- 3 **Goal-based agents**
- 4 Utility-based agents

Intelligent agents (recap)

- AI (modern view): The study and construction of **rational agents**
rational = **act** rationally = achieve the **best outcome**
- Rationality can be defined mathematically: select an action that maximizes its **utility** (a.k.a. **performance measure**), given
 - 1 the evidence provided by the percept sequence received so far and
 - 2 whatever built-in knowledge the agent has.

Some important kinds of agents:

- 1 Simple reflex agents
- 2 Model-based reflex agents
- 3 **Goal-based agents**
- 4 Utility-based agents

Goal-based agents are designed to reach a goal (e.g., a taxi driver must reach a particular destination) by finding sequences of actions.

- The fields of AI devoted to this problem are **searching** and **planning**

In this lecture we study goal-based agents which solve this problem by searching. They are also known as **problem-solving agents**.

Examples of searching problems



8 Queen



Hanoi Tower



Maze



15 puzzle

Problem-solving agents

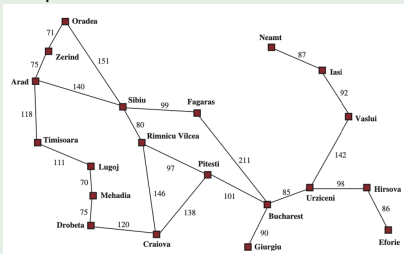
We will describe various algorithms to solve searching problems.

We will assume that:

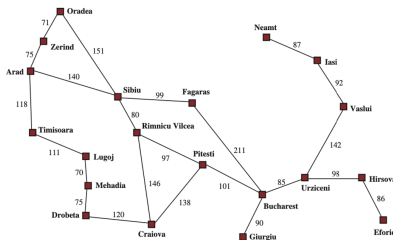
- The **states** of the world have atomic representations: they are identified by names and have no visible internal structure.
- The **environment** is episodic, single agent, fully observable, deterministic, static, discrete, and known.

Example (A route-finding agent)

An agent is in Arad (current state) and must go to Bucharest (goal) to catch a flight next day. He must find a sequence of actions (walk from town to town) to reach the goal. The agent has a map of Romania.



The four-phase problem-solving process: example



- 1 **Goal formulation:** reach Bucharest.
- 2 **Problem formulation:** describe the available states and actions available from every state
 - states = city names: {Arad, Sibiu, ...}
 - actions: $actions(Arad) = \{to(Sibiu), to(Oradea), to(Timisoara)\}, \dots$
- 3 **Search:** use simulation to find a sequence of actions that reaches the goal. Such a sequence is called a **solution**
Example: $to(Sibiu), to(Fagaras), to(Bucharest)$
- 4 **Execution:** execute the actions in the solution, one at a time.

Search problems and solutions

Formal definitions

A formal search problem consists of

- **State space** = the set of possible states that the environment can be in.
- **Initial state** = the state that the agent starts in. Example: Arad.
- A set of one or more **goal states**, by defining $\text{IsGoal}(s)$ if s is a goal state.
- The **actions** available to the agent. Given a state s , $\text{Actions}(s)$ returns a finite set of actions that can be executed in state s .
- A **transition model** which describes what each action does. $\text{Result}(s, a)$ returns the state that results from doing action a in state s .
Example: $\text{Result}(\text{Arad}, \text{to}(\text{Timisoara})) = \text{Timisoara}$.
- An **action cost function** $\text{Action-Cost}(s, a, s')$, or $c(s, a, s')$: the numeric cost of applying action a in state s to reach state s' .

A **solution** is a sequence of actions which forms a path from the initial state to a goal state. It is **optimal** if its cost minimum among all possible solutions.

Remark

The state space can be represented as a **directed graph**: the nodes are the states, and the directed edges between them are actions.

Formal descriptions of search problems

Example 1: Sudoku

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

- **States:** any valid Sudoku board
- **Initial state:** partially filled board
- **Action:** fill with a valid number
- **Goal:** fill the entire board and respect the problem constraints
- **Cost:** 1/move

- **State:** any valid Sudoku board
- **Initial state:** partially filled board
- **Goal:** fill the entire board and respect the problem constraints
- **Cost:** 1/move

Formal descriptions of search problems

Example 2: 8-Puzzle

5		8
4	2	1
7	3	6

STATE(N)

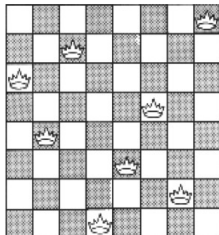
1	2	3
4	5	6
7	8	

Goal state

- **States:** the position of each digit on the grid
- **Action:** empty cell is moving to right, left, up and down
- **Testing the goal:** current state is equal with final state
- **Cost:** 1/action

Formal descriptions of search problems

Example 2: 8-Queens



- **States:** any configuration in which the queens do not attack each other
- **Action:** add a new queen on the leftmost unoccupied column such that it is not attacked by another queen
- **Testing the goal:** 8 queens are on the table
- **Cost of solution:** 0
- 2957 possibilities

Search algorithms

Given a search problem

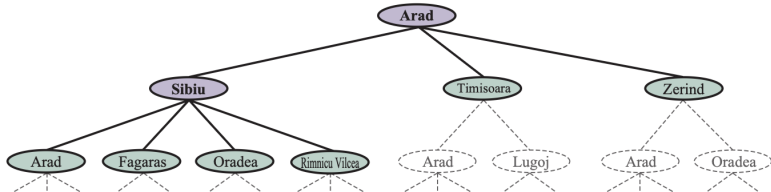
Return a solution, or an indication of failure if there is no solution.

We consider algorithms that superimpose a **search tree** over the state-space graph

- The root of the tree corresponds the initial state of the search problem
- The edges in the search tree correspond to actions.

We try to find a path from the root to a node corresponding to a goal state.

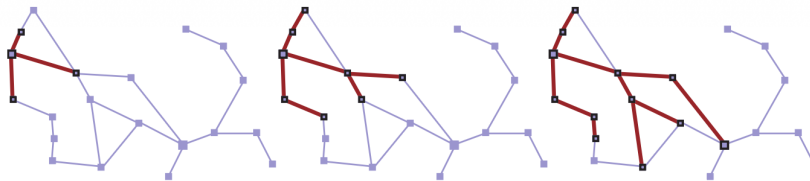
Example:



The construction of search trees

By repetitive **expansions** of a leaf node, until (hopefully) we reach a node for a goal state.

Illustrated example



- **Expansion of a node for s :** consider the $\text{Actions}(s)$ available for s , using the $\text{Result}()$ function to see where those actions lead to, and generating a new node (called child or successor) for each of the resulting states.
- **Essence of search:** Which node to expand next?

Best-first search

- Chooses a node, n with minimum value of some evaluation function, $f(n)$.

```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
  node  $\leftarrow$  NODE(STATE=problem.INITIAL)
  frontier  $\leftarrow$  a priority queue ordered by f, with node as an element
  reached  $\leftarrow$  a lookup table, with one entry with key problem.INITIAL and value node
  while not IS-EMPTY(frontier) do
    node  $\leftarrow$  POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    for each child in EXPAND(problem, node) do
      s  $\leftarrow$  child.STATE
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
        reached[s]  $\leftarrow$  child
        add child to frontier
  return failure
```

```
function EXPAND(problem, node) yields nodes
  s  $\leftarrow$  node.STATE
  for each action in problem.ACTIONS(s) do
    s'  $\leftarrow$  problem.RESULT(s, action)
    cost  $\leftarrow$  node.PATH-COST + problem.ACTION-COST(s, action, s')
    yield NODE(STATE=s', PARENT=node, ACTION=action, PATH-COST=cost)
```

Search data structures

node = data structure with 4 components:

- *node.State*: the state to which the node corresponds;
- *node.Parent*: the node in the tree that generated this node
- *node.Action*: the action applied to the parent's state to generate this node
- *node.Path-Cost*: we use $g(\text{node})$ as a synonym for Path-Cost.

Following the Parent pointers back from a node allows us to recover the states and actions along the path to that node. Doing this from a goal node gives us the solution.

- The generated nodes which are not yet expanded form the **frontier** of the search tree.
- The nodes in the frontier are stored in a data structure called **queue**, which provides the following operations:
 - $\text{Is-Empty}(\text{frontier})$ returns true only if there are no nodes in the frontier.
 - $\text{Pop}(\text{frontier})$ removes the top node from the frontier and returns it.
 - $\text{Top}(\text{frontier})$ returns (but does not remove) the top node of the frontier.
 - $\text{Add}(\text{node}, \text{frontier})$ inserts node into its proper place in the queue.

Different search algorithms use different kinds of queues: priority queue, FIFO queue, or LIFO queue.

- Best-first search stores the frontier nodes in a priority queue.

Redundant paths. Measuring the performance

We distinguish 2 kinds of redundant paths:

- ① **loopy path**: it has different nodes for the same state. Example: Arad-Sibiu-Arad
- ② Paths to a state with **higher cost** than other paths to the same state.
 - Example: Arad-Sibiu (140 miles long) versus Arad-Zerind-Oradea-Sibiu (297 miles long).
The second path to Sibiu is redundant because it is longer.

Redundant paths should not be considered when searching for optimal paths.

Redundant paths. Measuring the performance

We distinguish 2 kinds of redundant paths:

- ① **loopy path**: it has different nodes for the same state. Example: Arad-Sibiu-Arad
- ② Paths to a state with **higher cost** than other paths to the same state.
 - Example: Arad-Sibiu (140 miles long) versus Arad-Zerind-Oradea-Sibiu (297 miles long).
The second path to Sibiu is redundant because it is longer.

Redundant paths should not be considered when searching for optimal paths.

The performance of a problem-solving algorithm can be measured in 4 ways:

- ① **Completeness**: Is the algorithm guaranteed to find a solution when there is one, and to correctly report failure when there is not?
- ② **Cost optimality**: Does it find a solution with the lowest path cost of all solutions?
- ③ **Time complexity**: How long does it take to find a solution? This can be measured in seconds, or more abstractly by the number of states and actions considered.
- ④ **Space complexity**: How much memory is needed to perform the search?

Redundant paths. Measuring the performance

We distinguish 2 kinds of redundant paths:

- 1 **loopy path**: it has different nodes for the same state. Example: Arad-Sibiu-Arad
- 2 Paths to a state with **higher cost** than other paths to the same state.
 - Example: Arad-Sibiu (140 miles long) versus Arad-Zerind-Oradea-Sibiu (297 miles long).
The second path to Sibiu is redundant because it is longer.

Redundant paths should not be considered when searching for optimal paths.

The performance of a problem-solving algorithm can be measured in 4 ways:

- 1 **Completeness**: Is the algorithm guaranteed to find a solution when there is one, and to correctly report failure when there is not?
- 2 **Cost optimality**: Does it find a solution with the lowest path cost of all solutions?
- 3 **Time complexity**: How long does it take to find a solution? This can be measured in seconds, or more abstractly by the number of states and actions considered.
- 4 **Space complexity**: How much memory is needed to perform the search?

Remark. Some search algorithms have an infinite state space. To be **complete**, they must eventually reach any state that is connected to the initial state. A search algorithm with this property is called **systematic**.

Search strategies

There are 2 kinds of search strategies: uninformed and informed.

- An **uninformed** search algorithm has no clue how close a state is to the goal(s).

The most important uninformed search algorithms are: Breadth-first, Uniform-cost, Depth-first, Depth-limited, Iterative-deepening, and bidirectional (if applicable)

- An **informed** search algorithm uses domain-specific hints about the location of goals, The hints come in the form of a **heuristic function**, denoted $h(n)$:

- $h(n)$ = estimated cost of the cheapest path from the state at node n to a goal state.

In general, an informed (heuristic) search strategy can find solutions more efficiently than an uninformed strategy.

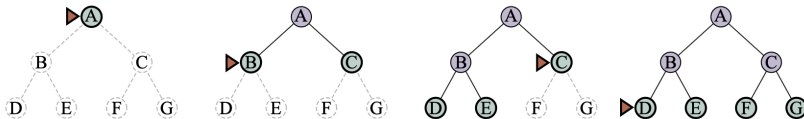
Typical informed search algorithms: Greedy Best-first, A*, IDA*

Breadth-first search (BFS)

Characteristics

- root node is expanded first, then all the successors of the root node are expanded next, then their successors, etc.
- This is a complete and systematic search strategy
- It coincides with Best-First-Search where $f(n) = \text{node depth}$ (i.e., number of actions needed to reach that node), but it can be improved with a number of tricks

Example (Breadth-first search on a simple binary tree)



Breadth-first search

Pseudocode

```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure  
  node  $\leftarrow$  NODE(problem.INITIAL)  
  if problem.IS-GOAL(node.STATE) then return node  
  frontier  $\leftarrow$  a FIFO queue, with node as an element  
  reached  $\leftarrow$  {problem.INITIAL}  
  while not IS-EMPTY(frontier) do  
    node  $\leftarrow$  POP(frontier)  
    for each child in EXPAND(problem, node) do  
      s  $\leftarrow$  child.STATE  
      if problem.IS-GOAL(s) then return child  
      if s is not in reached then  
        add s to reached  
        add child to frontier  
  return failure
```

Breadth-first search

Characteristics and performances

If the search tree has branching factor b and depth d then

- Complete search strategy
- All actions have default cost 1
- Total number of nodes generated is

$$1 + b + b^2 + \dots + b^d = O(b^d)$$

- All the nodes remain in memory
 - $\Rightarrow$ time and space complexity is $O(b^d)$
- The biggest problem of BFS is its **space complexity (exponential)**

Uniform-cost search

(a.k.a. Dijkstra's algorithm)

- BFS ignored the costs of actions, by assuming that all of them have cost 1.
- Uniform-cost search does not ignore them: it coincides with Best-First-Search where the cost function is the path cost

function UNIFORM-COST-SEARCH(*problem*) **returns** a solution node, or *failure*
return BEST-FIRST-SEARCH(*problem*, PATH-COST)

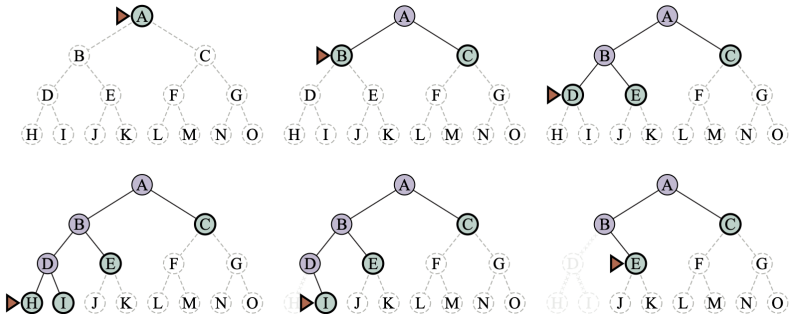
Properties of Uniform-cost search

- complete
- cost-optimal
- worst-case time and space complexity is $O(b^{1+\lceil C^*/\epsilon \rceil})$ where
 - C^* : cost of an optimal solution
 - $\epsilon > 0$: lower bound on the cost of each action

Depth-first search (DFS)

- Always expands the deepest node in the frontier first.
- It could be implemented as a call to BEST-FIRST-SEARCH where the evaluation function f is the negative of the depth.

Example (DFS on a simple binary tree)



Depth-first search

Characteristics and performances

- It is not systematic: it may run forever in an infinite state space without finding existing solutions
⇒ DFS is **incomplete**
- If the search tree has max. depth m and branching factor b , then
 - memory complexity is $O(bm)$ – **highly desirable!**
 - a variant of DFS, called **backtracking search**, has memory complexity $O(m)$

Depth-first search

Characteristics and performances

- It is not systematic: it may run forever in an infinite state space without finding existing solutions
⇒ DFS is **incomplete**
- If the search tree has max. depth m and branching factor b , then
 - memory complexity is $O(bm)$ – **highly desirable!**
 - a variant of DFS, called **backtracking search**, has memory complexity $O(m)$
- **Question:** Can we modify DFS to become a complete search strategy?

Depth-first search

Characteristics and performances

- It is not systematic: it may run forever in an infinite state space without finding existing solutions
⇒ DFS is **incomplete**
- If the search tree has max. depth m and branching factor b , then
 - memory complexity is $O(bm)$ – **highly desirable!**
 - a variant of DFS, called **backtracking search**, has memory complexity $O(m)$
- **Question:** Can we modify DFS to become a complete search strategy?
- **Answer:** **Yes**; see **Depth-limited** and **iterative deepening search**

Depth-limited and iterative deepening search

Depth-limited search (main idea): supply a depth limit, ℓ and treat all nodes at depth ℓ as if they had no successors

Pseudocode

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution node or failure  
  for depth = 0 to  $\infty$  do  
    result  $\leftarrow$  DEPTH-LIMITED-SEARCH(problem, depth)  
    if result  $\neq$  cutoff then return result
```

```
function DEPTH-LIMITED-SEARCH(problem,  $\ell$ ) returns a node or failure or cutoff  
  frontier  $\leftarrow$  a LIFO queue (stack) with NODE(problem.INITIAL) as an element  
  result  $\leftarrow$  failure  
  while not IS-EMPTY(frontier) do  
    node  $\leftarrow$  POP(frontier)  
    if problem.IS-GOAL(node.STATE) then return node  
    if DEPTH(node) >  $\ell$  then  
      result  $\leftarrow$  cutoff  
    else if not IS-CYCLE(node) do  
      for each child in EXPAND(problem, node) do  
        add child to frontier  
  return result
```

Performances of uninformed search algorithms

Summary

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ¹	Yes ^{1,2}	No	No	Yes ¹	Yes ^{1,4}
Optimal cost?	Yes ³	Yes	No	No	Yes ³	Yes ^{3,4}
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

Informed search strategies

Greedy best-first search

Greedy best-first search = a form of best-first search that expands first the node n with the lowest $h(n)$ value — the node that appears to be closest to the goal — on the grounds that this is likely to lead to a solution quickly. Thus, $f(n) = h(n)$.

- For the search problem of a route from Arad to Bucharest, $h(n)$ could be the straight-line distance between n and Bucharest.

Properties:

- complete in finite state spaces, but not in infinite ones.
- it is not necessarily optimal
- time and space complexity is $O(|V|)$ where V is the state space.

Informed search strategies

A*

The evaluation function of A*-search is $f(n) = g(n) + h(n)$ where

- $g(n)$: the path cost from the initial state to node n
- $h(n)$: estimated cost of the best path that continues from n to a goal.

Properties of A*-search

- complete, if we assume (1) the existence of $\epsilon > 0$ s.t. all action costs are $> \epsilon$, and (2) the state space either has a solution or is finite.
- cost-optimal, if the heuristic is admissible

A heuristic function $h(n)$ is

- **admissible** if for all nodes n :
 $h(n) \leq$ minimum cost from n to a final goal.
- **consistent** if, for every node n and every successor n' of n generated by an action a : $h(n) \leq c(n, a, n') + h(n')$

Remark: every consistent heuristic is admissible.

Heuristic functions

- ▶ They affect search performance
- ▶ How are they discovered?

Case study: the 8-puzzle. Its actions are described by:

- ▶ A tile can move from square X to square Y if X is adjacent to Y and Y is blank

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

The shortest solution is 26 actions long.

Some admissible heuristics:

- h_1 = number of misplaced tiles (blank not included).
For this example, all eight tiles are out of position, so the start state has $h_1 = 8$.
- h_2 = sum of the distances of the tiles from their goal positions. Because tiles cannot move along diagonals, the distance is the sum of the horizontal and vertical distances – sometimes called the **city-block distance** or **Manhattan distance**.
For this example, $h_2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$.

How to generate heuristic functions?

1. From relaxed problems

Relaxed problem = problem with fewer restrictions on the actions

- ⇒ The state graph of the relaxed problem is a supgraph of the state graph of the initial problem.
 - supergraph of a graph = graph with extra edges
- ⇒ the cost of an optimal solution to a relaxed problem is a consistent heuristic for the original problem.

Examples of relaxed problems of the 8-puzzle:

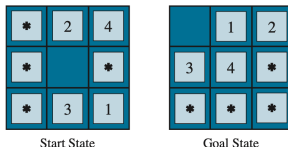
- (a) A tile can move from square X to square Y if X is adjacent to Y.
- (b) A tile can move from square X to square Y if Y is blank.
- (c) A tile can move from square X to square Y.

From (a), we can derive h_2 From (c), we can derive h_1 .

How to generate heuristic functions?

2. Generating heuristics from subproblems

Example of a subproblem of the 8-puzzle:



The cost of the optimal solution of this subproblem is a lower bound on the cost of the complete problem.

⇒ Main idea: build a **pattern database** to store exact solution costs for every possible subproblem instance

Example: $9 \cdot 8 \cdot 7 \cdot 6 \cdot 5 = 15120$ patterns for every possible configuration of the four tiles and the blank ⇒ the heuristic $h_{DB}(s)$ is obtained by lookup search of the subproblem configuration corresponding to state s in the database.

How to generate heuristic functions?

3. Heuristics with landmarks

Main idea = **precomputation** of some optimal path costs between pairs of vertices.

- There are $|V| \cdot (|V| - 1)$ pairs $\Rightarrow O(|E|^3)$ time and $O(|V|^2)$ space to compute and store them all.
 - impractical for large state spaces ($|V| > 10^5$)
- Better approach: choose a small number (15 – 29) of **landmark** points; for each vertex v and landmark point L :
 - precompute and store the optimal costs $C^*(v, L)$ and $C^*(L, v)$

We can define the following heuristics:

- $$h_1(n) := \min_{L \in \text{Landmarks}} (C^*(n, L) + C^*(L, \text{goal}))$$

h_1 is efficient but inadmissible

- $$h_{DH}(n) := \max_{L \in \text{Landmarks}} |C^*(n, L) - C^*(\text{goal}, L)|$$

h_{DH} is called a **differential heuristic**. It is both efficient and admissible.

Chapter 3 of

- S. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th edn. (Pearson Education Ltd., 2021).